Computer Organization and Architecture Lab (CE) **Spring 2025**

Project Description

Marks: 100 (weight: 25 %)

You will work in a team of 2 students.

Title of the Project:

Design, Simulation & Testing of a 5-Stage Pipelined RISC-V processor using Verilog HDL

Introduction:

RISC-V is a modern open-source Instruction Set Architecture (ISA) that supports scalable processor design. In this project, you will extend your existing Single-Cycle RISC-V processor into a **basic 5-stage pipelined processor**, implementing instruction flow optimization and improving performance.

This project provides practical exposure to pipelined processor design, inter-stage communication, and simulation of multiple concurrent instructions.

Project Requirements:

You are required to:

- Implement a 5-stage pipelined RISC-V processor:
  - IF (Instruction Fetch)
  - ID (Instruction Decode/Register Fetch)
  - EX (Execute)
  - MEM (Memory Access)
  - WB (Write Back)
- Insert pipeline registers between stages.
- Handle basic data hazards using **stalling** or **forwarding**.
- Simulate and verify execution of simple RISC-V instruction sequences.

Students can optionally add branch handling (bonus marks!).

Project Phases:

Phase 1: Pipelined Datapath Design

- Modify the existing Single-Cycle Datapath into a pipelined datapath.
- Insert pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB).
- Modify control signals as necessary.

Computer Organization and Architecture Lab (CE) Spring 2025

Phase 2: Hazard Handling

- Implement basic **data forwarding** (preferred) or **stalling**.
- Explain clearly how hazards are handled in the report.

Phase 3: Simulation and Testing

- Write a simple test program with at least 8–10 instructions covering:
  - ALU operations
  - Load/Store
  - Branch (optional for bonus)
- Verify correct execution by observing register/memory values at each cycle.

Phase 4: Documentation

- Submit a project report including:
  - Block diagram of the pipelined datapath.
  - Description of pipeline stages.
  - Verilog code snippets for major changes.
  - Hazard handling mechanism description.
  - Simulation results and analysis.

Deliverables:

- Verilog source files (datapath, control, pipeline registers).
- Assembly test program.
- Verilog testbench.
- Simulation waveforms (.png or screenshot).
- Complete Project Report (5–8 pages).

Computer Organization and Architecture Lab (CE) **Spring 2025**

How to Start?

Phase I: Build a Single-Cycle processor (deadline: 9th May 2024)

[After design, you will need to verify that it executes a given subset of the MIPS instruction set. **40 Marks** are reserved for this single-cycle processor design and its verification].

Specification of Single-Cycle Processor

Implement the needed components in Verilog. **You may use behavioral or a combination of behavioral and structural Verilog features**. All Verilog models should correspond to realistic components, such as, registers, comparators, shifters, etc.

No super-composite components, e.g., a branch unit that takes in the opcode, operands, and PC, and outputs a new PC, or something similar to that. You will be given a number of basic datapath components that you *may* use or extend. All storage elements are negative edge triggered.

Datapath Components

You should start by designing these sub-components (some of these have been provided as references):

- **ALU** **32-bit ALU**,
- **Extender** **sign/zero extender**,
- **Multiplexers** **m16x2, m32x2, m32x3, m32x5, m5x2**: multiplexers with various widths and number of inputs,
- **Register** **reg32**: 32-bit register,
- **Register File** **regfile**: register file,
- **Shifter** **Arithmetic logical, multi-bit shifting unit.**
- **Instruction Memory** 2K-words deep, 32-bit wide, (using "SRAM"), initial content will be loaded from within the Verilog code.
- **Data Memory** 4K-words deep, 32-bit wide, no initial contents.

There should be an arithmetic/logical multi-bit (32) shifter *external* to the ALU. Furthermore, instructions such as slt should have effect outside the ALU, as specified in the textbook.

`slt $rd,$rs,$rt` should subtract the two operands, and then generate the appropriate result value for the destination register. Your data path should then store this result back in the destination register.

Component Propagations Delays

All of your Verilog components must incorporate "reasonable" propagation delays. For each component, think about how that component would be implemented using discrete gates. Use this mental discrete gate implementation to estimate the delays to use. There is no exact right or wrong answer for the delays to use. However, if the delays are too large or too small, you will be asked to justify your decision. The delay models may be kept simple, i.e., one delay value can be used for an entire component (which should be the maximum worst-case delay). For the datapath controller you should use a delay of 20*T*.

Please look through the Verilog code of the provided components for examples of how to specify propagation delays for the various components and how to support edge-triggered clocking. You should try to use reasonable delays for the blocks with storage elements.

Resetting and Initializing the Datapaths

Your datapaths must employ an active-low, master reset signal, called Reset L. This signal should be kept at the de-asserted level at all times, except when the processor is being reset. To reset the processor, drive Reset L low for at least 10 integral clock cycles and then reset Reset L to high level. The master reset should feed into all reset types of signals of the datapath, such as those of the register file. You have to provide a unit that is external to the main control units that contains at least one register called IPC. This register supplies the initial Program Counter (PC) that your processor should execute after `reset`. IPC and all other input control signals should be driven by the top level test module of your model.

Testing and Diagnostic Programs

You should write diagnostic programs to test your MIPS processor. These programs should exercise all instructions your datapath supports. Your program should be written such that the result of the test is "displayed" on the STDOUT. In general, you can display the internal state of a module by using appropriately coded `$display("...");` statements from within the module. For instance, you can use `$display("Reg[rt]=%b...",Reg[rt])` statements in a module to print on the STDOUT the contents of a register `rt`. It is important to write good diagnostic programs to test your hardware because it will simplify your work later on when you have to debug a more complicated processor.

Single-Cycle Data path

Develop an implementation in Verilog of a single-cycle processor for a subset of the MIPS instruction set given below.

Type	Instructions
arithmetic (unsigned)	**`addu, subu, addiu`**
arithmetic (signed)	**`add, sub, addi`**
Logical	**`and, andi, or, ori, xor, xori`**
Shift	**`sll, sra, srl`**

Compare	**slt, slti, sltu**
Control	**beq, bne, blez,**
data transfer	**lw, sw,**

Use the actual MIPS machine language instruction formats, given in the MIPS reference data sheet.

Verilog Controller for the Datapath

Build the Control Unit (CU) for the single cycle datapath. You may use any behavioral Verilog instruction available to develop the CU. This CU should take the exact bits from the 32-bit instruction and generate the control signals. You must add any control signals necessary to read instructions off the instruction memory, and the "next instruction logic" to generate the Program Counter (PC) of the instruction which needs to be fetched next. The Verilog code that specifies the behavior of your CU must be well structured. Use the Verilog `` `define SYMBOL code_to_substitute `` as much as possible and specify upper bounds for units as `parameter n = ....` **Note:** Recall that in the Single-Cycle Processor, everything should take place in one clock cycle T_{CC}. Make sure that you are using sufficiently (but not excessively) long clock period T_{CC}, for your Single-Cycle Processor. *Turn in a copy of the output matrix for the control unit, processor schematic, diagnostic programs, and a simulation log that shows the correct execution of the test MIPS programs.*

You will have to include detailed justification for the selection of T_{CC} in your Single-Cycle Processor. You need to be prepared to demonstrate evidence for this justification, in terms of longest combinational circuit paths in your design. **Hint**: Use Verilog to measure the times it takes to compute signal propagations within your datapath.

Phase II: Pipeline your single-cycle processor of Phase I. Verify that it executes the given subset of the MIPS instruction set. **(deadline: 19th May 2024)**

[**60 Marks** are reserved for pipelined processor and its verification]

Pipelining

You have to implement the following five-stage pipeline:

- **IF (Instruction Fetch):** Access the instruction cache for the instruction to be executed.
- **ID (Instruction Decode):** Decode the instruction and read the operands from the register file. For branch instructions, calculate the branch target instruction address and compare the registers.
- **EX (Execute):** Execute on the operands from the previous stage. One of the following activities can occur:

1. For computational instructions, the ALU executes arithmetic or logical instructions.
2. For load or store instructions, the data address is calculated.

- **MEM (Data Memory Access):** Access the data cache for load or store instructions.
- **WB (Write Back):** Write result to register file.

Add the appropriate pipeline registers to your single cycle design. *Assume that memory accesses take only one cycle in your implementation.*

Initial Testing

To test your program, you should write diagnostic programs. The general structure of the programs should be some calculations, data manipulations, etc., followed by `sw`'s to the main memory. Then, at points during the execution and at the end of the program, you can dump memory to show that the correct values were correctly written to memory.

Keep in mind that you haven't handled hazards yet, so you must be careful that your test programs don't try to use values too soon after they are generated.

Pipelining Speedup (or Gain)

Calculate the cycle time for your pipelined processor. In your write-up, compare the cycle time from Phase I to that of Phase II.

Deliverables

Every member of the project group is required to keep an independent design notebook that documents his/her design process and all the steps in the program development; it should document the progress that you made while writing your software (**keep track of date/time**). Write the design notebook while you are developing the software, do not write on it afterwards.

- **Verilog Code**
- **Design notebook of each group member**
- **Complete Project Report**

Submit the above required materials on the classroom on or before **(deadline: 19th May 2024)**

NOTE:

- **Ensure that your program is clearly commented**.
- **Submit your test programs too**. **Failure to do so will result in a penalty on your marks.**
- **Every individual member will be assessed independently based on the work done by him.**
- **Provide the reference in your documentation of modules you got from any other resource.**
- **Any type of plagiarism if found, will result in zero marks.**